

CSE447 Lab Project - Setup Guide

Project Overview

This implementation includes all required features:

1.  Login/Register system
2.  User info encryption (AES-256 for database fields)
3.  Password hashing with SHA-256 + salt
4.  Separate credential check function
5.  Key management module (CryptoManager)
6.  Post encryption/decryption using RSA
7.  Complete database encryption
8.  MAC integrity checking (HMAC-SHA256)

Algorithms Used

- **RSA-2048**: For post encryption (asymmetric)
- **AES-256-CBC**: For database field encryption (symmetric)
- **SHA-256**: For password hashing with salt
- **HMAC-SHA256**: For MAC integrity verification

Prerequisites

1. **Python 3.7+**
2. **MongoDB** (local installation or cloud)

Installation Steps

1. Install MongoDB

Ubuntu/Debian:

```
bash

sudo apt-get update
sudo apt-get install mongodb
sudo systemctl start mongodb
```

macOS:

```
bash  
  
brew install mongodb-community  
brew services start mongodb-community
```

Windows: Download from MongoDB official website and follow installation guide.

2. Install Python Dependencies

```
bash  
  
pip install -r requirements.txt
```

3. Run the Application

```
bash  
  
python secure_app.py
```

File Structure

```
project/  
├── secure_app.py    # Main application file  
├── requirements.txt # Python dependencies  
├── aes_master.key   # Auto-generated AES key (don't delete!)  
└── README.md        # This file
```

Security Features Explained

1. Password Security

- SHA-256 hashing with 32-byte random salt
- Salt stored separately from hash
- Constant-time comparison for verification

2. RSA Encryption for Posts

- 2048-bit RSA key pair per user
- Posts encrypted with each user's public key
- Only users with private keys can decrypt

3. Database Encryption

- All sensitive fields encrypted with AES-256
- Separate master key for database encryption
- IV (Initialization Vector) used for each encryption

4. Integrity Protection

- HMAC-SHA256 for data integrity
- Prevents tampering with user data and posts
- Verified on each read operation

Testing the Application

Basic Test Flow:

1. **Register** two users (e.g., "alice" and "bob")
2. **Login** as alice
3. **Post** a message as alice
4. **Logout** and login as bob
5. **View posts** - bob should see alice's encrypted post decrypted

Expected Behavior:

- Passwords are never stored in plaintext
- Database contains only encrypted data
- Posts are readable only by authenticated users
- MAC verification ensures data integrity

MongoDB Collections Structure

Users Collection:

json

```
{
  "username": "alice",
  "email_encrypted": "base64_encrypted_email",
  "password_salt": "base64_salt",
  "password_hash": "sha256_hash",
  "private_key_encrypted": "base64_encrypted_private_key",
  "public_key": "pem_public_key",
  "mac": "hmac_sha256",
  "created_at": "timestamp"
}
```

Posts Collection:

```
json
{
  "author": "alice",
  "encrypted_content": {
    "alice": "rsa_encrypted_for_alice",
    "bob": "rsa_encrypted_for_bob"
  },
  "mac": "hmac_sha256",
  "created_at": "timestamp"
}
```

Key Management

- RSA keys: Generated per user, private key encrypted in database
- AES master key: Generated once, stored in `aes_master.key`
- Salts: Random 32-byte salt per password
- MACs: HMAC-SHA256 using master key

Demonstration Points

1. Show encrypted data in MongoDB
2. Demonstrate user registration and login
3. Show post creation and viewing
4. Explain key management
5. Show MAC verification working
6. Demonstrate that database breach reveals no plaintext data

Troubleshooting

MongoDB Connection Issues:

- Ensure MongoDB is running: `sudo systemctl status mongod`
- Check connection string in code (default: localhost:27017)

Cryptography Errors:

- Ensure latest cryptography library: `pip install --upgrade cryptography`

Key File Issues:

- Don't delete `keyfile` - it's needed for decryption
- If lost, you'll need to re-register all users